

PA ATOMIC

Food Hygiene Prototype: Plan & Workflow

A demo-only build to prove the three-layer model on a single data source.

Objectives

The prototype has three jobs:

- **Prove the concept works.** A working demo that takes real FSA food hygiene data and runs it through all three layers (Daily Feed, Custom Alerts, AI Anomaly Detection) end-to-end.
- **Be convincing to stakeholders.** Polished enough that leadership and pitching event attendees can see the vision, not just the code. PA-branded, clean outputs, a clear story arc.
- **De-risk the full build.** Surface the technical and editorial decisions that will matter at scale — anomaly sensitivity, alert formatting, deduplication logic — so they can be refined before a bigger commitment.

What the prototype is NOT: production infrastructure, a real subscription product, or a system that any real newsroom depends on. Reliability, security, and scalability are deliberately deferred.

Scope

Data Source

Food Standards Agency Food Hygiene Rating Scheme (FHRS) API. No authentication required, returns JSON, covers England, Scotland, Wales and Northern Ireland. Proven pipeline pattern from your RADAR FSA work.

Geographic Focus

Start with a small set of local authorities — Dundee plus 3–5 neighbouring Scottish authorities, or equivalent test region. Narrow scope makes the prototype tractable and the demo outputs meaningful (not overwhelming). Expansion to national coverage is a post-prototype concern.

Layers Delivered

All three: Daily Feed, Custom Alerts, and AI Anomaly Detection. Each must work end-to-end, but depth is prioritised over feature completeness. One well-executed anomaly detector is better than three half-working ones.

Out of Scope

- Real subscriber management, authentication, or billing
- National coverage (deferred until post-prototype)

- Cross-dataset correlation with PA wire coverage (flagged as future capability but not built)
- Self-service alert configuration UI — rules live in config files for the prototype
- Mobile or native apps — email and web only

Architecture Overview

Five components, each doing one job well:

Component	Purpose
1. Ingestion	Daily pull from FSA FHRS API for selected local authorities. Stores every record with timestamp.
2. Change Detection	Differs today's pull against yesterday's. Produces a clean 'what changed' log: new establishments, rating changes, re-inspections, score shifts.
3. Layer Engine	Feeds the change log into three parallel processes: Daily Feed (digest generation), Custom Alerts (rule evaluation), Anomaly Detection (statistical analysis).
4. Delivery	Renders output as PA-branded HTML emails and a static demo webpage. Email sent to test inboxes; webpage hosted on GitHub Pages or similar.
5. Demo Layer	A public-facing webpage showing what the product does: live establishment count, latest digest, recent alerts, current anomaly flags, and a how-it-works explainer.

Tech Stack

- **Language:** Python. Strongest fit for API handling, scheduling, email delivery, and the anomaly detection layer (pandas, numpy, scikit-learn).
- **Storage:** SQLite for the prototype. Zero setup, version-controllable, easily migrated to PostgreSQL later if needed.
- **Scheduling:** GitHub Actions on a daily cron. Reuses the deployment pattern already working for the Teams notifications and Innovation Digest.
- **Email delivery:** A transactional email service (Mailgun, SendGrid, or even a simple SMTP relay) for the prototype. Test recipients only.
- **AI layer:** Claude API for natural-language summarisation of anomaly flags. Statistical detection itself in Python using classical methods (z-scores, moving averages, fuzzy string matching).
- **Demo page:** Static HTML hosted on GitHub Pages. Regenerated by the daily GitHub Action.
- **Branding:** PA palette (purple, yellow, charcoal). Greycliff CF with Helvetica fallback. Reuses the design patterns from the Innovation Digest.

Workflow

What happens, in order, each day the prototype runs:

1. Ingestion

A scheduled job queries the FHRS API for each of the selected local authority IDs. For each establishment returned, the full record is written to the SQLite database with today's pull date. Existing records from previous pulls remain intact — the database accumulates a history of every version of every record, not just the latest state.

2. Change Detection

A diff process compares today's records against the most recent previous version of each establishment. Four categories of change are extracted:

- **New establishments:** FHRS ID not seen before.
- **Rating changes:** Same FHRS ID, different rating value from last known record.
- **Re-inspections (no change):** Same FHRS ID, same rating, new rating date.
- **Score changes:** Same FHRS ID, same overall rating, but hygiene/structural/management sub-scores shifted.

The output is a JSON change log for the day — the atomic unit all three layers consume.

3. Daily Feed Generation

The change log is rendered into a PA-branded HTML email. Structure:

- Header with date and coverage area
- 'Zero-rated premises today' section (most newsworthy, shown first)
- 'Significant drops' section (2+ star reductions)
- 'New inspections' section (all other rating changes)
- 'Improvements' section (ratings that went up)
- Each entry: business name, address, local authority, old rating, new rating, inspection date, link to FSA ratings page

If no changes are detected on a given day, the digest simply notes this and confirms the pipeline ran successfully — journalists need to trust that silence means silence, not failure.

4. Alert Rule Evaluation

A YAML config file defines alert rules. Each rule has a name, a condition type, parameters, and a recipient (for the prototype, recipient is always a test inbox). The engine evaluates every rule against the change log. If a rule matches, a dedicated alert email is generated immediately, separate from the daily digest.

Four rule types supported for the prototype:

- **Watchlist:** Alert when inspection data changes for a specific list of FHRS IDs (e.g. a newsroom's local notable premises).
- **Threshold:** Alert when any premises in a target local authority drops by N or more rating points.
- **Zero-rating:** Alert for any new 0-rated premises in a target area.

- **Business type:** Alert when any school kitchen, hospital caterer, or care home receives a new rating.

Each alert email has a clear, filterable subject line: 'PA Atomic Alert: [rule name] — [summary]'.

5. Anomaly Detection

Runs after change detection, on both today's changes and a rolling 90-day window of historical changes. Three detectors for the prototype:

- **Chain-wide pattern:** Uses fuzzy matching on business names to group likely branches of the same chain. Flags when a chain's rolling-average rating drops significantly or when multiple branches receive low ratings within a defined window.
- **Geographic cluster:** Flags when a single local authority shows an unusually high number of low ratings (zero or one star) within a short window, compared to its own historical rate.
- **Rapid re-inspection:** Flags premises re-inspected unusually quickly after a low rating — suggests active FSA monitoring and a potential follow-up story.

Each flag is passed to the Claude API with context (the raw data, the historical baseline, the pattern detected) and a structured prompt asking for a plain-English summary suitable for a journalist. The output is a flag card: headline, plain-English explanation, underlying data, and a confidence indicator.

Flags are surfaced on the demo webpage and in a weekly 'Anomaly Digest' email (separate from the daily feed).

6. Demo Page Regeneration

After all three layers have run, a static HTML page is regenerated and pushed to the demo hosting. The page shows the current state of the system — what it's monitoring, what it's found, what it would alert on — and serves as the central artefact for the pitching event.

Suggested Build Order

Build in a sequence that keeps you with a working end-to-end system at every stage. Even a minimal pipeline is more valuable than a half-built sophisticated one.

#	Milestone	What you have at the end
1	Ingestion + Storage	Daily pull from FHRS API for your chosen local authorities, stored with timestamps in SQLite. Manually verify the data looks right.
2	Change Detection	Clean daily change log in JSON. Test it by manually injecting a 'previous' version and running the diff — does it catch everything it should?
3	Daily Feed Email	PA-branded HTML digest arrives in a test inbox every morning. Even if empty, it runs reliably. This alone is a credible product.
4	One Alert Rule	Zero-rating alert working end-to-end. Pick the simplest rule first to prove the pattern; the others follow the same template.
5	Remaining Alert Rules	Watchlist, threshold, and business-type alerts added. YAML config makes adding rules easy once the engine works.
6	Chain-Wide Anomaly	First and most visually impressive detector. Fuzzy-matched chains, statistical flagging, Claude summarisation.
7	Other Anomaly Detectors	Geographic cluster and rapid re-inspection detectors added.
8	Demo Webpage	Static HTML page on GitHub Pages. Shows system state, latest outputs, how-it-works explainer. Regenerated by the daily GitHub Action.
9	Polish	Brand consistency, subject-line tightening, email HTML rendering tested across clients, demo page copy refined for non-technical audiences.

Watchpoints

A few things worth paying attention to early, based on the RADAR experience and the FSA API's known quirks:

- **FSA API behaviour:** The API works server-side but not from browsers (CORS issues). Fine for this prototype since everything runs in GitHub Actions, but worth noting for any future client-side ambitions. Include the version header on every request.
- **Chain detection is the hard part:** Business names in FHRS aren't standardised — 'McDonald's', 'McDonalds Restaurants', 'McDonald's Restaurant Ltd' all appear. Plan to iterate on fuzzy matching and potentially maintain a manual canonical-name mapping for major chains.
- **Baseline data need:** Anomaly detection needs history to work. Either backfill several months of FHRS data at setup or pull history via the API before running the live detectors. The FSA API returns current state only — historical comparison has to come from your own pulls.

- **Demo honesty:** Be clear on the demo page what's automated and what's still manual. A prototype misrepresented as a product damages credibility. Framing it as 'working proof of concept' is stronger than pretending it's further along than it is.
- **Claude API costs:** Keep anomaly-flag summarisation batch-based rather than per-record. Costs will be minimal at prototype scale but worth being deliberate from the start.
- **False positive logging:** Build a simple mechanism to mark flags as 'not a story'. Even if no real newsroom is using the prototype, you can do this yourself during demo prep — and the logged data becomes evidence of editorial oversight for the pitch.

What Success Looks Like

At the end of the prototype build, you should be able to demonstrate:

- A live pipeline that pulls FSA data daily without intervention
- A PA-branded daily digest email showing real changes in the target area
- At least one real alert that has fired during development (even if you had to wait for a relevant rating change)
- At least one anomaly flag with a Claude-generated plain-English explanation
- A polished demo webpage you can screen-share during the pitching event
- A clear story about what the full version would do that this prototype doesn't

That last point matters. The prototype's job is not to be the full product — it's to make the full product imaginable. Keep a running list of 'this is what scale would add' (national coverage, more data sources, newsroom self-service, cross-dataset correlation with PA wire copy) so you can speak confidently to the roadmap when someone asks.

Demo-only. Narrow scope. One data source. All three layers. Convincing enough to imagine the rest.